Date: 2016-05-08
Thomas Köppe <tkoeppe@google.com>

Comma omission and comma deletion

ISO/IEC JTC1 SC22 WG21 P00306r1
To: EWG, CWG, WG14 liaison

ISO/IEC JTC1 SC22 WG14 N2044
To: WG21 liaison

Contents

Revision history

- WG21 P00306r0: Initial proposal.
- WG14 N2034: Same as P00306r0, but addressed to WG14.
- WG21 P00306r1/WG14 N2044: This version. Changed proposed wording for `__VA_OPT__` from P00306r0/N2034 to be clear about balanced parentheses and resulting tokens; added alternative "`#define F(X...)`".

Note to WG14: This paper is being proposed to WG21, and the section numbering and some of the examples make superficial reference to C++. The wording and the problem are identical in C, though, with the section mapping described at the end.

Summary

This is a proposal to make variadic macros easier to use with no arguments by adding a new special functional macro `__VA_OPT__`.

The problem

Function-style macros that can have variable arguments suffer from a number of ill-specified corner cases. Consider the following macro definitions:

```
#define F(X, ...) f(10, X, __VA_ARGS__)
#define G(X)      f(10, X)
#define H(...)    f(10, __VA_ARGS__)
```

Invocations of these macros are surprising:

Invocation	Effect	Notes
`F(a, b, c)`	`f(10, a, b, c)`	variable arguments are "`b, c`"
`F(a, )`	`f(10, a, )`	variable arguments contain zero tokens syntax error
`F(a)`	ill-formed	violates 16.3p12 (no variable arguments)
`G(a)`	`f(10, a)`	
`H(a, b, c)`	`f(10, a, b, c)`	variable arguments are "`a, b, c`"
`H(a)`	`f(10, a)`	variable arguments are "`a`"
`H()`	`f(10, )`	variable arguments are "" syntax error

There are two problems:

1. When the macro definition ends in `...)`, the invocation must contain at least as many commas as the macro has mandatory parameters. This makes the invocation `F(a)` invalid.
2. In the case of zero mandatory parameters, it is not possible to distinguish between zero arguments in the invocation and one (variable) argument that is empty. The root of this problem is the unfortunate human convention by which a list of n elements is presented with $n - 1$ infix separators. This convention does not degenerate to the case $n = 0$, and empty lists are always a special case.

However, it is quite natural for a macro invocation with variable arguments to degenerate to the case where there are no arguments. In the example, we would like `F(a)` to be replaced with `f(10, a)`. A more realistic example is a custom diagnostic facility such as the following:

```
#define ERROR(msg, ...) std::printf("[" __FILE__ ":%d] " msg, __LINE__, __VA_ARGS__)

ERROR("%d errors.\n", 0);  // OK, std::printf("[" "file.cpp" ":%d] " "%d errors.\n", 7, 0);
ERROR("No errors.\n");     // Error
```

The complication arises when we consider `H()`. We may perhaps wish it to be replaced with `f(10)`. However, we may also wish to have a macro such as

```
#define ADD_COMMA(...) , __VA_ARGS__
```

which always produces a comma, even when invoked with no arguments. The difference is that we consider `H()` to have zero arguments, whereas we consider `ADD_COMMA()` to have one, empty argument.

Defining the goals

We would like to make the preprocessor more expressive to allow users to write macros for all of the situations described above. This requires two distinct changes, one simple and the other complex.

Goal 1. Allow the omission of the comma before the variable arguments in the invocation (i.e. allow `F(a)` rather than requiring `F(a, )`).

Goal 2. Provide a mechanism to express a replacement text that contains the variable arguments but which contains a separating comma only if the variable arguments are non-empty (i.e. allow both `f(10, a)` and `f(10, a, b)` as possible replacements of `F`). At the same time, continue to provide a mechanism that unconditionally contains comma before the (possibly empty) variable arguments, like `ADD_COMMA` above.

This behaviour of Goal 1 is already supported by many popular compilers as a non-conforming extension. It is a non-breaking change, since the current syntax `F(a)` is ill-formed. Goal 2 is much harder to solve, since there is no single simple enhancement of the existing semantics that satisfies all possible use cases.

We will step through a series of possible solutions (inspired by existing vendor extensions) and analyse their shortcomings, before presenting the proposed solution.

Alternative designs

1. Delete any commas if there are no variable arguments

This approach does not add any new syntax. It merely solves Goal 1 above by allowing a variadic macro invocation to not contain any variable arguments. However, under this approach, the absence of variable arguments is taken as a request to *delete* an existing comma immediately preceding the `__VA_ARGS__` token:

```
F(a, )   =>   f(10, a, )   // has variable argument, replaced as-is
F(a)     =>   f(10, a)     // no variable arguments, final comma deleted
H()      =>   f(10, )      // has (empty) variable argument, replaced as-is
```

This is a minimal, unsurprising extension. However, it suffers from the major draw-back that it offers no mechanism to delete a trailing comma from a variadic macro with zero mandatory parameters.

A variant of this extension is currently provided by MSVC++ and Embarcadero compilers, which always delete the comma, even in the case of zero mandatory arguments. Another possible extension is to provide those semantics under a new name (e.g. `__VA_ARGS_FOO__`).

2. Hijack existing syntax to opt in to comma deletion

This approach also allows the omission of the variable arguments, and in addition it reuses the concatenation operator `##` to control comma deletion explicitly:

```
#define F1(X, ...) f(10, X, __VA_ARGS__)
#define F2(X, ...) f(10, X, ## __VA_ARGS__)

#define H1(...) f(10, __VA_ARGS__)
#define H2(...) f(10, ## __VA_ARGS__)

F1(a, b)   =>   f(10, a, b)   // standard
F1(a, )    =>   f(10, a, )    // standard (empty variable arguments)
F1(a)      =>   f(10, a, )    // extension (no variable arguments), final comma unaffected

F2(a, b)   =>   f(10, a, b)   // variable arguments not empty
F2(a, )    =>   f(10, a, )    // variable arguments present (though empty), final comma unaffected
F2(a)      =>   f(10, a)      // no variable arguments, final comma deleted

H1(a)      =>   f(10, a)      // standard
H1()       =>   f(10, )       // standard (empty variable arguments)

H2(a)      =>   f(10, a)      // variable arguments not empty
H2()       =>   f(10)         // variable arguments present (though empty), final comma deleted
```

This extension is somewhat difficult to explain, but it generally Does What You Want. The complete omission of variable arguments is required for comma deletion (compare `F2(a, )` and `F2(a)`), though omission of the variable arguments alone is not enough to delete the comma (compare `F1(a, )` and `F1(a)`), but the case of zero mandatory parameters is special, and in that case it is mere absence of tokens from the variable arguments that enables the comma deletion when the `##` operator is used.

The downside of this extension is three-fold: 1) Parsing this syntax requires look-ahead, adding cost to the translation. 2) The extension reuses an unrelated piece of syntax, muddling the language. 3) The extension hides its dependency on the presence or absence of the variable arguments and whether the variable arguments contain tokens in subtle and non-explicit ways.

3. “Named pack” style

A rather more different approach abandons the use of C99’s `__VA_ARGS__` token in favour of something like `#define F(X, Args...)` or `#define F(X, ...Args)`. GCC has long provided the former (where the replacement text would use `Args` for the variable arguments, and `, ##Args` (with mandatory whitespace after the comma!) requests comma deletion). The template-pack-like syntax `...Args` does not appear to be used by any preprocessor and may provide a less obstructed extension route (e.g. one could say that `x, y, ...Args` always has comma deletion semantics).

However, all these approaches seem undesirable. First off, they are a departure, and perhaps even a regression, from the direction taken by C99 and its `__VA_ARGS__` token. Second, this design would only satisfy those needs that require comma deletion, leaving use cases like the above `ADD_COMMA` to use the existing syntax. Thus there would be two parallel but dissimilar constructions living side by side, which seems inelegant and wasteful.

4. Omit comma from the macro definition

Note: This idea came up during the discussion of N2034 in WG14.

We could use syntax like `#define F(X ...) f(10, __VA_ARGS__)` to request comma deletion (note the absence of a comma before the ellipsis in the definition). This approach does not degenerate to the case of macros with no named parameters, though. Moreover, WG14 felt that this was too clever and too subtle, whereas the proposed solution below is highly visible and explicit. Also, an unrelated difference between C++ and C is that C++ allows omitting the final comma from the parameter list of a variable function declaration. While this has nothing to do with the preprocessor, the semantics of the optional comma of that feature are the opposite of this present consideration, which is unnecessarily confusing.

5. A new token

All of the considered extensions so far have in common that they end up creating a parallel set of constructions which are identical to the existing macro facilities except when the macro is invoked with no variable arguments, and they all provide some automatic mechanism to determine when to delete a comma. However, none of them are quite explicit about what they are doing.

For the next idea, we consider adding a new token. Let us call it `__VA_ARGS_OPT__`, with the semantics that wherever it appears in the replacement text, it is replaced with the variable arguments (just like `__VA_ARGS__`), but additionally, whenever the variable arguments do contain tokens, a comma is prepended:

```
#define F(X, ...) f(10, X __VA_ARGS_OPT__)

F(a, b)  =>   f(10, a, b)  // __VA_ARGS_OPT__ => ', b'
F(a, )   =>   f(10, a)     // empty variable arguments, __VA_ARGS_OPT__ => ''
F(a)     =>   f(10, a)     // empty variable arguments, __VA_ARGS_OPT__ => ''
```

In this approach, we have separated Goals 1 and 2 entirely; whether a leading (!) comma is inserted now only depends on whether the variable arguments contain tokens, not on whether they are present at all.

Discussion and proposal

We already said that solutions 1, 2 and 3 are ultimately inelegant, since they create a redundant structure that replicates existing facilities and only differs in subtle details. Solution 4 (a new token) feels cleaner and more orthogonal. In the words of Richard Smith:

“I remain unconvinced that implicitly adding or removing a comma is a good idea. We need the user to tell us which behavior they want.”

We can do a little better than solution 4. Our proposal is to add a new, special kind of *functional* macro `__VA_OPT__`. This macro may only be used in the replacement text of a variadic macro:

```
#define F(X, Y, ...) a b c __VA_OPT__(content)
```

The semantics are as follows: If the variable arguments contain no tokens, then `__VA_OPT__(content)` is replaced by no tokens (more precisely, by a placemarker). Otherwise, it is replaced by *content*, which can contain any admissible replacement text, including `__VA_ARGS__`.

The canonical use case of `__VA_OPT__` is for an optional separator:

```
#define LOG(msg, ...) printf(msg __VA_OPT__(,) __VA_ARGS__)

LOG("hello world")   // => printf("hello world")
LOG("hello world", ) // => printf("hello world")
LOG("hello %d", n)   // => printf("hello %d", n)
```

However, this mechanism allows other constructions, too:

```
#define SDEF(sname, S, ...) S sname __VA_OPT__(= { __VA_ARGS__ })

SDEF(foo);           // => S foo;
SDEF(bar, 1, 2, 3);  // => S foo = { 1, 2, 3 };
```

```
#define LOG(...)                                    \
    printf("at line=%d" __VA_OPT__(": "), __LINE__);  \
    __VA_OPT__(printf(__VA_ARGS__);)                \
    printf("\n")

LOG();                          // => printf("at line=%d", 123); printf("\n");
LOG("All well in zone %n", n);  // => printf("at line=%d: ", 123); printf("All well in zone %n", n); printf("\n");
```

Impact

The proposal is a pure extension of the preprocessor. Syntax that was previously not allowed becomes admissible under the proposed changes.

Implementation experience

The proposed extension to allow omission of the variable arguments has been implemented by many compilers. We do not know of any experience with anything resembling the `__VA_OPT__` extension.

Proposed wording

Change paragraph 16.3p4 as follows.

> If the *identifier-list* in the macro definition does not end with an ellipsis, the number of arguments (including those arguments consisting of no preprocessing tokens) in an invocation of a function-like macro shall equal the number of parameters in the macro definition. Otherwise, there shall be ~~more~~at least as many arguments in the invocation ~~than~~as there are parameters in the macro definition (excluding the `...`). There shall exist a `)` preprocessing token that terminates the invocation.

Change paragraph 16.3p5 as follows.

> The ~~identifier~~identifiers `__VA_ARGS__` and `__VA_OPT__` shall occur only in the replacement-list of a function-like macro that uses the ellipsis notation in the parameters.

Change paragraph 16.3p12 as follows.

> If there is a `...` immediately preceding the `)` in the function-like macro definition, then the trailing arguments (if any), including any separating comma preprocessing tokens, are merged to form a single item: the variable arguments. The number of arguments so combined is such that, following merger, the number of arguments is either equal to or one more than the number of parameters in the macro definition (excluding the `...`).

Append a new paragraph to subsection 16.3.1 as follows.

> The identifier `__VA_OPT__` shall always occur as part of the token sequence `__VA_OPT__(`*`content`*`)`, where *`content`* is an arbitrary sequence of *preprocessing-token*s other than `__VA_OPT__`, which is terminated by the closing `)` and skips intervening pairs of matching left and right parentheses. The token sequence `__VA_OPT__(`*`content`*`)` that occurs in the replacement list is replaced by a placemarker preprocessing token (16.3.3, 16.3.4) if the variable arguments consist of no tokens, otherwise it is replaced by *`content`*. [Example:
>
> ```
> #define F(...)             f(0 __VA_OPT__(,) __VA_ARGS__)
> #define G(X, ...)          f(0, X __VA_OPT__(,) __VA_ARGS__)
> #define SDEF(sname, ...) S sname __VA_OPT__(= { __VA_ARGS__ })
>
> F(a, b, c)       // replaced by f(0, a, b, c)
> F()              // replaced by f(0)
>
> G(a, b, c)       // replaced by f(0, a, b, c)
> G(a, )           // replaced by f(0, a)
> G(a)             // replaced by f(0, a)
>
> SDEF(foo);       // replaced by S foo;
> SDEF(bar, 1, 2);  // replaced by S bar = { 1, 2 };
> ```
>
> – end example]

Compatibility with C

The entire proposal (rationale, implementation experience and wording) applies almost verbatim to the C language as well. (For the wording changes, the C++ section 16.3 corresponds to the C section 6.10.3.) We would like to ask the WG14 liaison to discuss this proposal with WG14 and provide feedback, and we would like to encourage WG14 to adopt the same extension for C in the interest of future compatibility.

Acknowledgements

Many thanks to Dawn Perchik, David Krauss and Richard Smith for valuable discussion, guidance, suggestions, examples and review! Thanks also go to the members of WG14 for their hospitality and a very productive discussion.